

Everything is bits.

Numbers, negative numbers, overflow, characters, text, files,
images, code — one notation, and what the machine makes of it.

← ICS 02 · Size

BINARY

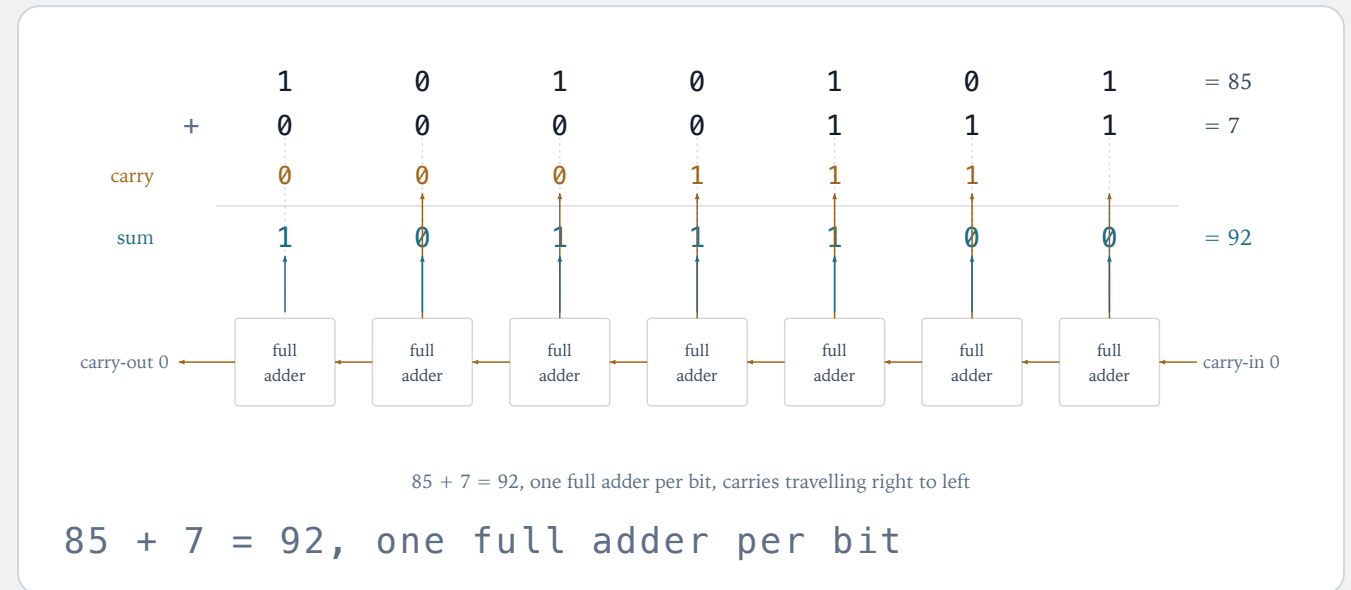
85 is **1010101**. Nothing else changed.

Decimal: $85 = 8 \cdot 10 + 5 \cdot 1$. Binary: $1010101 = 1 \cdot 64 + 0 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1$.

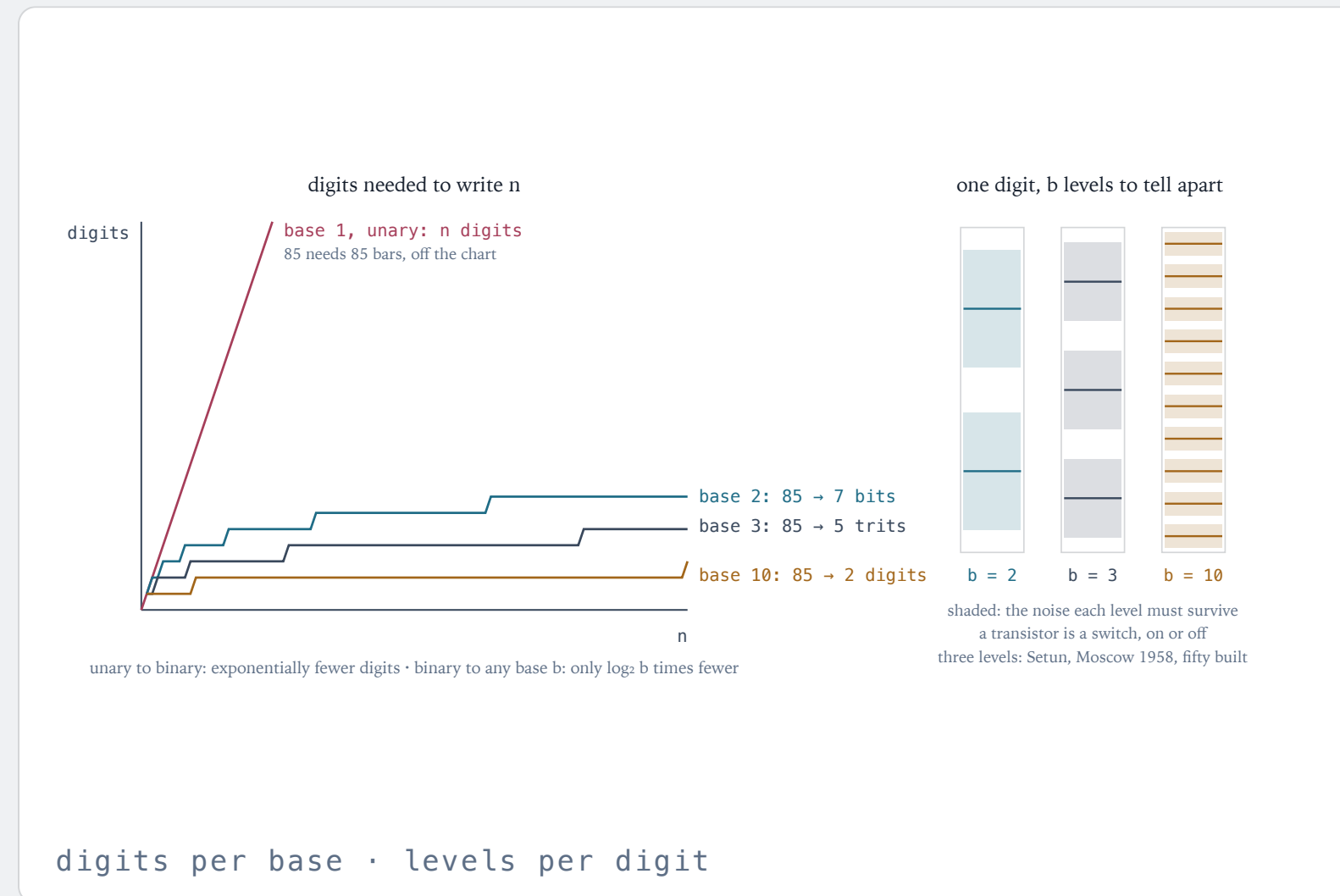
Same number, a different base. The value is the meaning; the digits are the notation.

Adding works the same way in any base — carry when a column overflows its base. $1010101 + 111 = 1011100$, which is $85 + 7 = 92$.

Hexadecimal groups four bits into one digit: 1010101 is 0x55. Octal groups three. Both are just shorter ways to write the same bits; the machine only ever sees the bits.



Above base one: **exponentially** shorter.
 Above base two: only a **constant factor**.



Unary writes n as n bars; positional notation needs $\lceil \log_b n \rceil$ digits: 85 is 85 bars, 7 bits, 2 decimal digits. Base 1 to 2 buys an exponential; base 2 to 10 buys $\log_2 10 \approx 3.3$, for every n .

The price is arithmetic. In unary, counting appends a bar: constant time. In a positional base an addition walks the digits: $\log n$ steps, with carries.

So the mathematics says leave unary, and does not say which base. That is for physics.

A transistor is a **switch**. Two levels leave the most room for noise, and nothing to calibrate.

B LEVELS PER DIGIT

A base- b digit must hold one of b distinguishable states. The gap between neighbouring levels is what noise has to cross, and it shrinks with b . Two levels, on and off, put that gap as far apart as the supply voltage allows.

COST PER LEVEL, NOT PER DIGIT

Hardware pays per level, not per digit. If a digit costs b and you need $\log_b n$ of them, $b / \ln b$ is smallest near e : 3 beats 2 by 5 percent. Five percent buys no factory; a switch that needs no calibration does.

WHERE HIGHER BASES DO WIN

Flash memory stores 2 to 4 bits per cell as 4 to 16 charge levels: a constant-factor saving, paid for in error correction, and read back out as bits.

And DNA uses base 4. Its digits are molecules, not voltage levels: four distinct shapes, each pairing with exactly one other, so four states cost nothing in noise margin, and pairing is how a strand is copied and checked. A different physics, a different base.

Boolean algebra lets the rest of the machine forget the physics: two values, and every gate a function of them.

Balanced ternary: digits $-1, 0, 1$. Negation is a **digit flip**, and no sign bit is needed.

Write base 3 with digits $-1, 0$ and 1 . Every integer, negative or not, has exactly one representation, and $-n$ is n with every digit negated: $85 = 81 + 3 + 1$ is $1\ 0\ 0\ 1\ 1$, and -85 is $-1\ 0\ 0\ -1\ -1$. No two's complement, no sign convention.

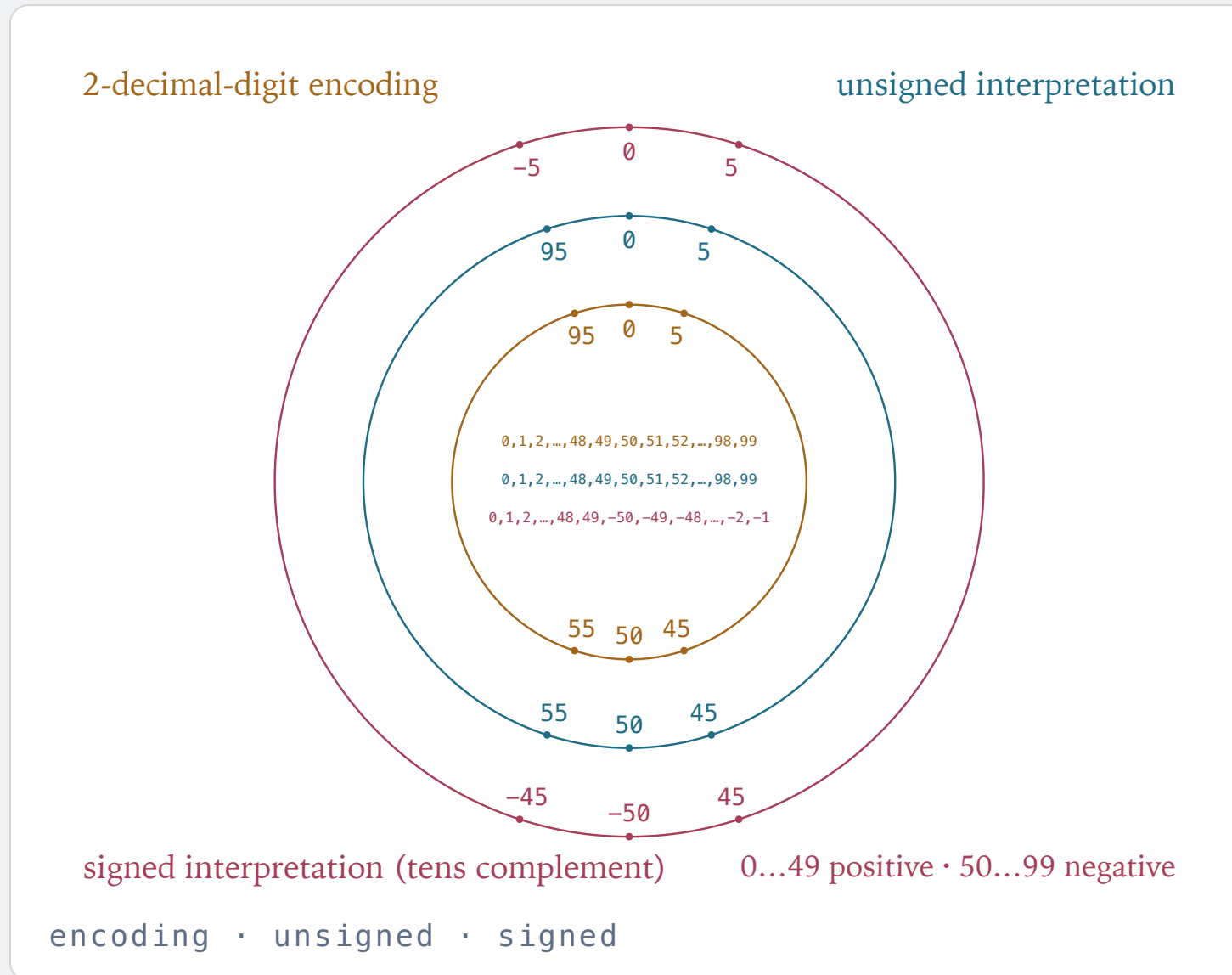
Setun, built in Moscow in 1958, computed in balanced ternary. Knuth called it the prettiest number system of all. It lost anyway, not to a better idea but to the transistor, which is a switch.

	BINARY	BALANCED TERNARY
digits	0 1	-1 0 1
negatives	convention	built in
negation	invert, add 1	flip digits
length	$\log_2 n$	$0.63 \cdot \log_2 n$
levels	2	3

Mathematically the nicer system; physically three levels where two would do. The transistor decides.

NEGATIVE NUMBERS

Two digits, read three ways.



With two decimal digits you can encode a hundred things. Read them as 0 to 99, or read 50 to 99 as -50 to -1 . Same digits. Subtraction becomes addition: $7 - 8$ is $7 + 92 = 99$, which is -1 .

The machine does exactly this in base two: two's complement. With 64 bits, 2^{64} encodings, read as 0 to $2^{64}-1$ or as -2^{63} to $2^{63}-1$. The bits do not say which.

1010101 with seven bits is 85 unsigned — and -43 signed. Same seven bits.

Finite means the numbers wrap around.

With 64 bits, $2^{64} - 1$ plus 1 is 0. Nothing crashes; the carry out of the top bit is simply lost. That is an overflow, and the machine does not tell you.

Signed, the same wrap makes $2^{63} - 1$ plus 1 equal to -2^{63} : the largest positive number plus one is the most negative one.

Every integer in every program you will ever run lives in a finite box. Most of the time nobody notices. The times somebody does are famous.

```
$ cat examples/overflows.c
// ... prints UINT64_MAX + 1, INT64_MAX + 1,
// and what happens
$ make selfie.h
$ ./selfie -c selfie.h \
    examples/overflows.c -m 1
...
```

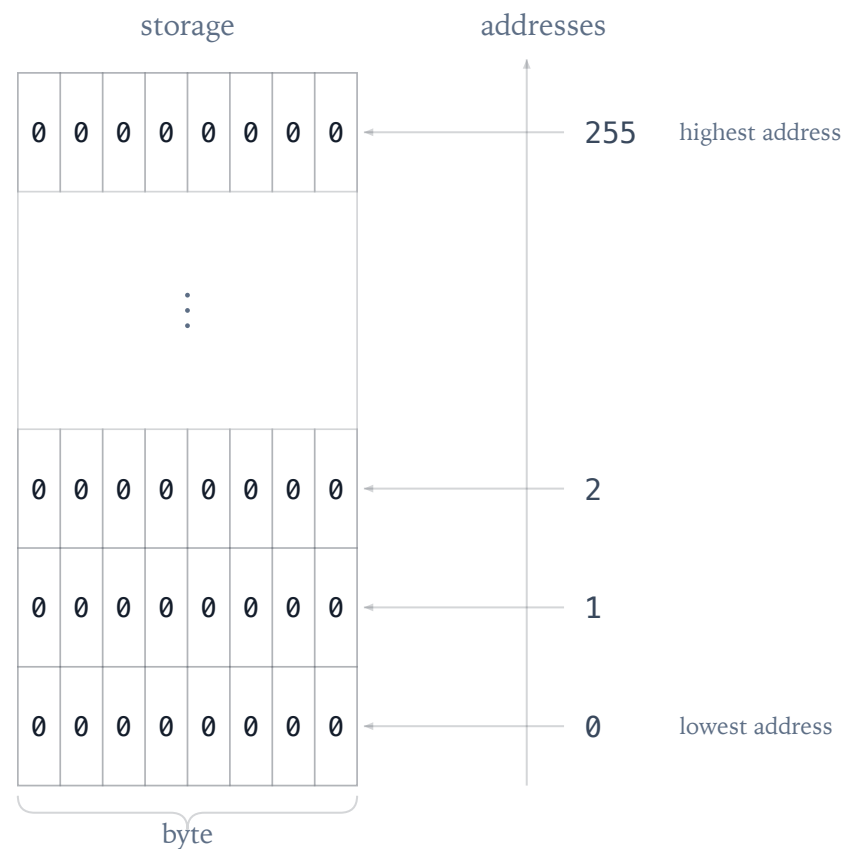
1010101 is also the letter **U**. People in the 1960s sat down and agreed.

ASCII: seven bits, 128 characters, a table agreed in 1963. 85 is U. 48 is the digit 0, 65 is A, 97 is a, 32 is a space. The digit '0' and the number 0 are different things with different codes.

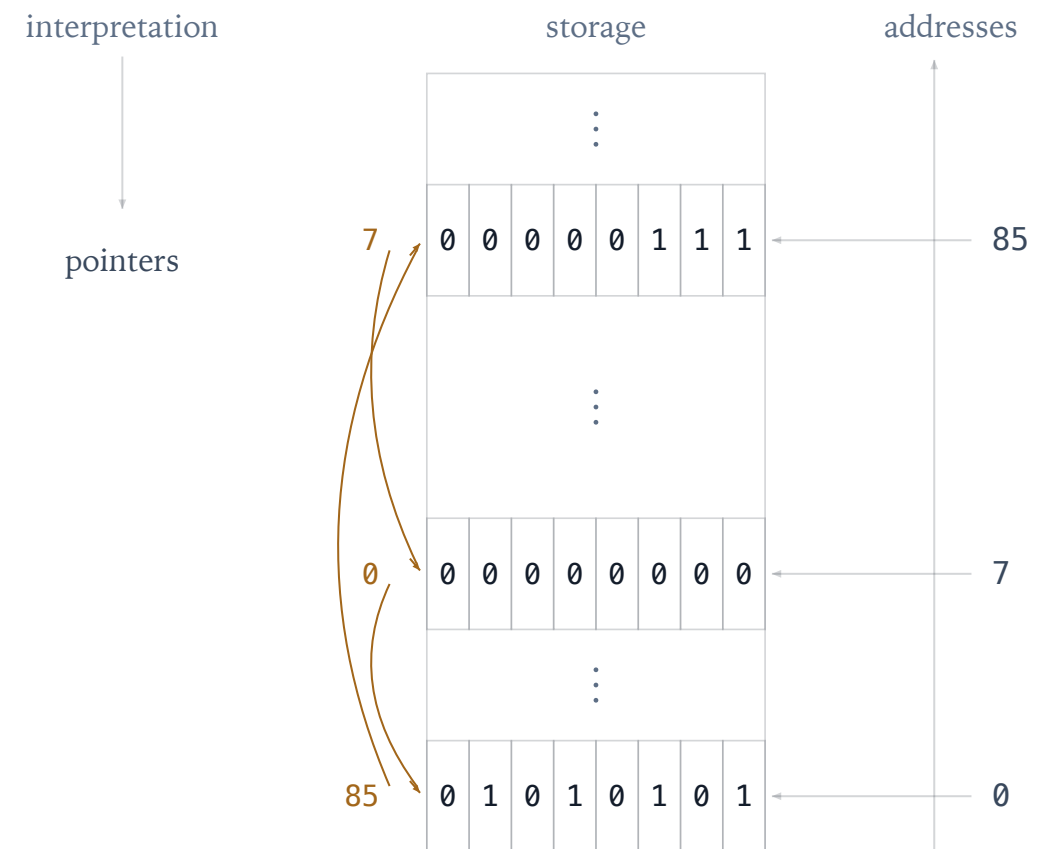
Unicode extends the table to every script, and UTF-8 encodes it in one to four bytes so that ASCII is unchanged. A byte is eight bits, a nibble four, and the figure names its ends.



Memory is bytes with **addresses**. An address is a number. So a byte can hold an address.

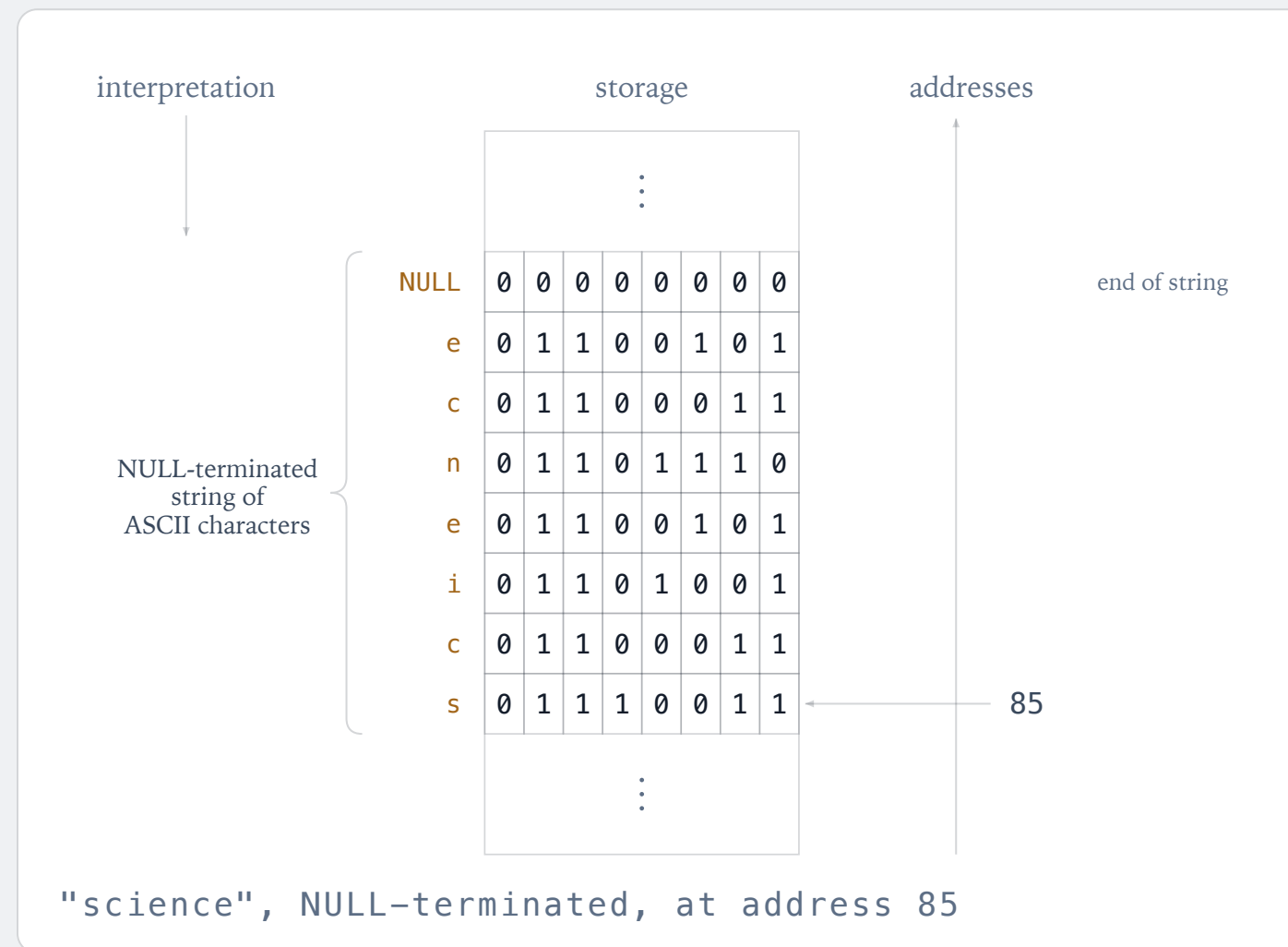


storage and addresses



a value read as an address: a pointer

A string is bytes in a row, ended by a **zero**. A file is a string with a name.

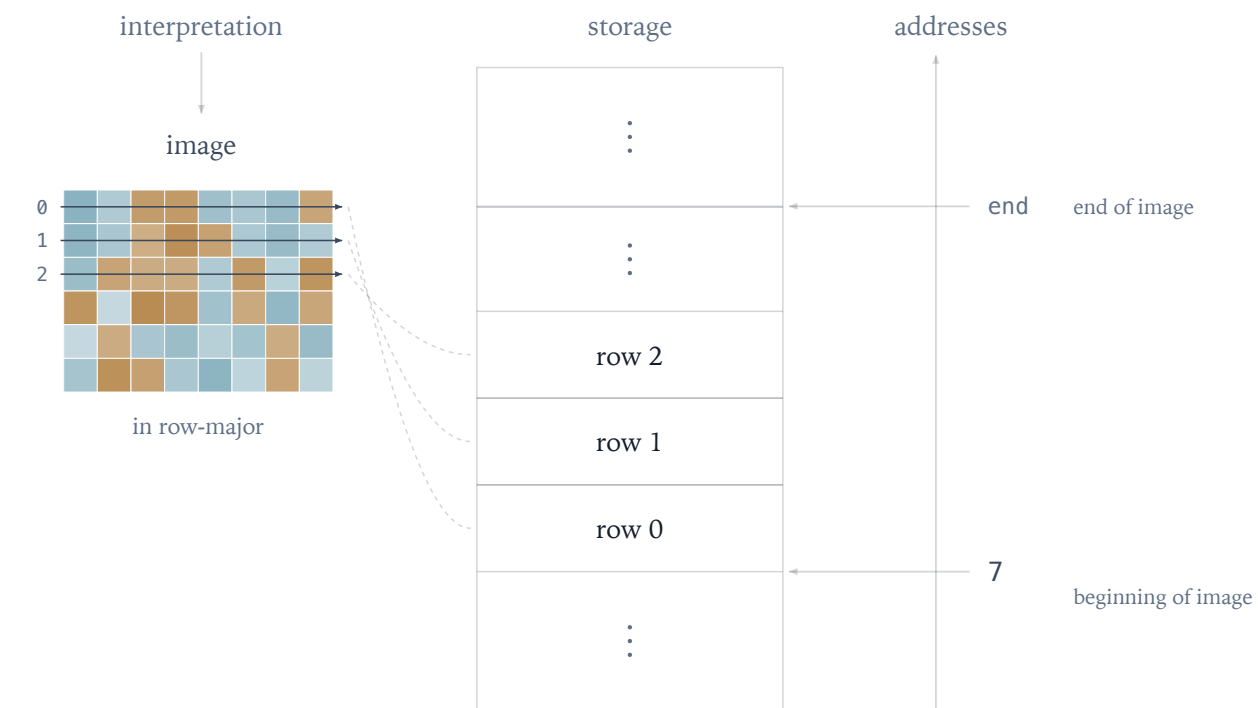


Contiguous: the letters of a word one after the other. The zero byte says where it ends, which is how a machine that only sees bytes knows a word is over.

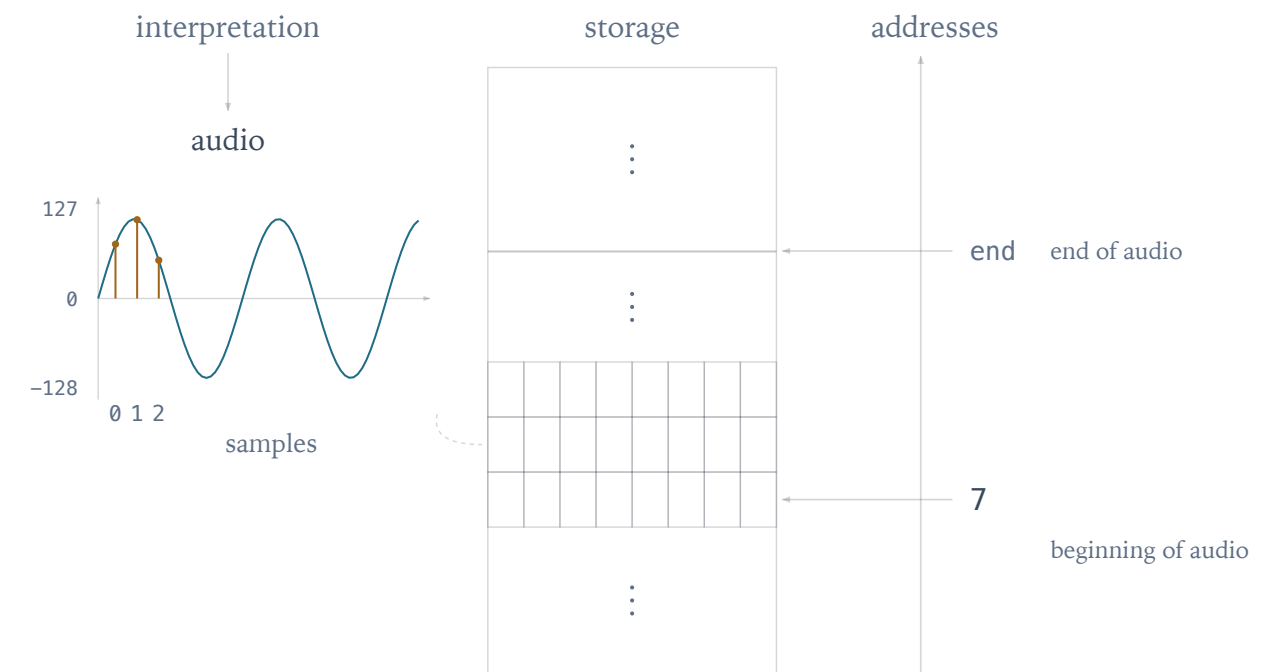
Non-contiguous: a text is paragraphs, each contiguous, reached through pointers. A directory is names and pointers to files. A file system is a tree of directories, and a pathname is the route from the root.

All of it is bytes at addresses. The structure is in how they are read.

A picture is numbers in rows. A film is pictures in a row. A sound is numbers in a row.

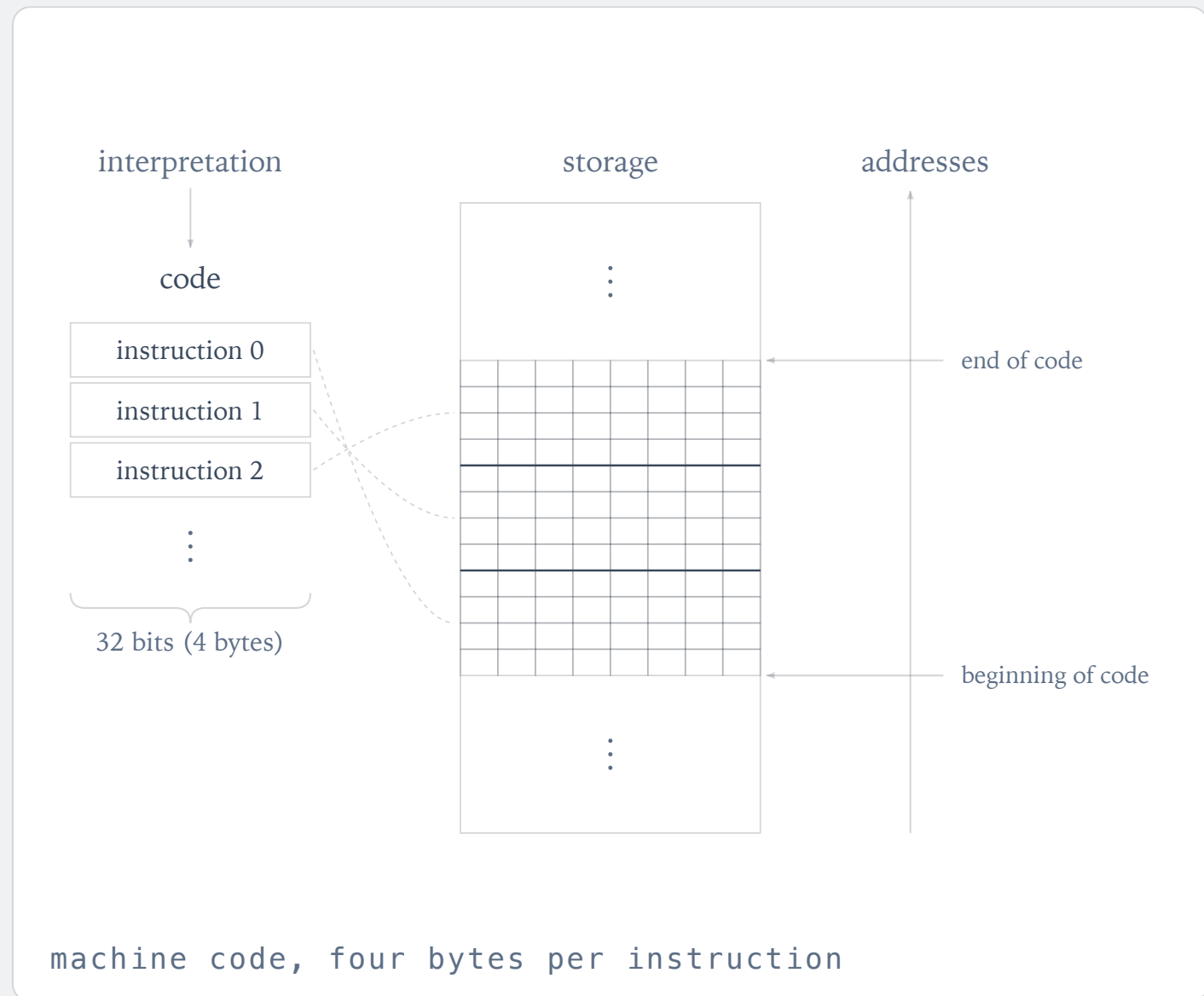


an image, row-major



audio, 8-bit samples

And an **instruction** is 32 bits. Code is bytes too.



Selfie compiles its source to 43,492 instructions of 32 bits each: 173,968 bytes of code and 14,424 bytes of data, laid out one after the other in a file.

That file is bytes at addresses, like a text or an image. The machine reads it and does what it says. Nothing in the bytes tells you whether they are code or data; the machine chapter shows what does.

```
$ ./selfie -c examples/hello-world.c -m 1
./selfie: selfie compiling examples/hello-world.c
to 64-bit RISC-U with 64-bit starc
...
Hello World!
```

1010101 is 85, and U, and -43 , and an address, and half an instruction. The bits do not say which.

Everything a machine stores is a number.
Everything a number means is decided by
whoever reads it, and by nothing else.

Last week: one prefix, two meanings. This
week: one byte, five. The gap between the
notation and its meaning is not a defect of
computers. It is what this class is about, and
next week we start on the notations that pin
meaning down.

*Whatever you see on a screen could
come from anywhere and mean
anything.*

THE BOOK, LIFE 4

Recommended exercises.

- 1 Read the Size chapter from *Numbers to Code*.
- 2 Write 42, 255, and 1000 in binary and hexadecimal. Add 85 and 42 in binary with carries.
- 3 How many digits does 1000 need in unary, binary, ternary and decimal? Which ratios stay the same as the number grows?
- 4 What is 1010101 as a signed 7-bit number? What is 11111111 as an unsigned and as a signed byte?
- 5 Write your first name in ASCII, in binary, and count the bits. Then in UTF-8 if it has an umlaut.
- 6 Run `examples/overflows.c` and explain each line of output.
- 7 In the pointers figure, what if the byte at address 0 held 7 instead of 85?

NEXT WEEK

Notation: formal languages. EBNF, and the two languages selfie is made of — C* with seven keywords, RISC-U with fourteen instructions — read exactly.

Listen, then read.

LISTEN

[Mozart · Symphony No. 40 in G minor](#) — Harnoncourt, Concentus Musicus. One two-note sigh, the smallest unit, and the whole first movement is that figure at different positions. Everything is bits, in G minor.

READ

[Code: The Hidden Language of Computer Hardware and Software](#) — Petzold, second edition 2022: from flashlights and Morse code to a working computer, one bit at a time. And, more technical, [A Mathematical Theory of Communication](#) — Shannon, 1948: where the bit got its name.